

Entities + Assumptions

There are 8 entities in the ER diagram:

Player: Represents a basketball player for the NBA. It is its own entity, as a player can appear in multiple teams throughout their career and play multiple games. Players are uniquely identified by an integer id.

Team: Represents a basketball team for the NBA. It is its own entity, as a team can play multiple games, and visit multiple arenas. Teams are uniquely identified by an integer id.

Game: Represents a basketball game between two teams in the NBA. It is its own entity, as a game requires two players. Games are uniquely identified by an integer id.

Official: Represents an NBA basketball game referee. It is its own entity, as an official can officiate multiple NBA games throughout their career. Officials are identified uniquely through an integer id.

Arena: Represents the venue which an NBA basketball game was hosted. This entity has a one to many relationship with games, as for one given arena, many games can be played at it. Arenas are identified uniquely through an integer id.

PlayerGameStats: Represents players' statistics for a given game, including free throw percentage, rebounds, turnovers, etc. It is assigned as an associative entity as for each player who participates in a game, they have their own specific attributes given for that game. A player's game statistics are uniquely identified by their playerid and gameid, which are foreign keys referencing their respective tables.

TeamGameStats: Represents teams' statistics for a given game, including free throw percentage, rebounds, turnovers, as an entire team. It is assigned as an associative entity as for each team who participates in a game, they have their own specific attributes given for that game. A team's game statistics are uniquely identified by their teamid and gameid, which are foreign keys referencing their respective tables.

GameOfficial: Represents all of the officials who have refereed for a single NBA game, it is an associative entity, as many officials can referee many NBA games throughout their career. A game's referee is uniquely identified by their officialid and gameid, which are foreign keys referencing their respective tables.

Relationships + Cardinality

The following section is used to explain the relationships between entities in the ER diagram:

Arena to Game: Arena has a one to many relationship with Game, as a game is played at only ONE arena, but an arena can host many games. Thus, we describe its cardinality as one to many.

Game to TeamGameStats to Team: Given that for one game, there are two teams which play, they have collected their own game statistics for that game. This results in a many to many relationship, as a game has multiple teams, and a team can play multiple games. given that these statistics cannot fit in either Game or Team, a junction table is created to display these stats. For the three entities, a single game has many TeamGameStats (two per game), and a single team has many TeamGameStats. The assumption is that each game has exactly 2 rows in TeamGameStats.

Game to PlayerGameStats to Player: A single game contains many players who record their own individual stats such as free throw percentages, minutes, rebounds, etc. A single player will play in many games in their career. Thus, this is a many to many relationship between a Game and Player. Given a relationship has its own attributes, we create a junction table PlayerGameStats to track these statistics. For the three entities, a single Game has many PlayerGameStats (number of players that played in that game), and a single Player has many PlayerGameStats (dependent on number of games played.)

Game to GameOfficial to Official: Each game is officiated by multiple officials, and each official officiates many games. This is a many-to-many relationship between Game and Official, represented by the associative entity GameOfficial. A single Game has many GameOfficial rows, and a single Official has many GameOfficial rows.

Normalization

The following sections describes and shows why the following database entity-relationship diagram is in **3NF**.

Given the Player entity, all non-key attributes such as height, weight, name, are all uniquely identified by the playerid. Those attributes cannot be identified by anything else. Thus the player entity is in 3NF.

Given the Team entity, we see that it's only non-key attributes, name, and city, can only be identified by the teamId, thus the Team entity is in 3NF.

Given the Official entity, their fullName can only be identified by their unique id. Thus the Official entity is in 3NF.

Given the Game entity, their date and arenaId depend only on the gameId, no other attributes can derive them. Thus Game is in 3NF.

Given the Arena entity, their non-key attributes name, city, state can only be retrieved through the arenaId, thus Arena is in 3NF.

The TeamGameStats entity contains two attributes as its primary key: gameId and teamId. Though, each stat attribute in the relation is entirely dependent on the combination of which game and which team we're choosing. Thus, there are no partial dependencies, and there is no transitive dependencies. Thus, TeamGameStats is in 3NF.

The PlayerGameStats entity contains two attributes as its primary key: gameId and playerId. Given that each stats attribute depends on the full key (since a player's stats can only be determined by which game they've played). teamId identifies the team the player played for in that game. Team descriptive attributes (name/city) are stored only in Team, avoiding transitive dependencies. Thus, gameId and playerId is a superkey for every functional dependency, making PlayerGameStats 3NF.

For the GameOfficialEntity, it has no non-key attributes at all, thus making it vacuously BCNF, and thus making it 3NF.

We store only IDs as foreign keys and keep descriptive attributes in their own entity tables to avoid transitive dependencies. Since all entities are in 3NF, the entire schema is in 3NF.

Relational Schema Design

The following section describes the schema design in a relational schema design format:

- Player(playerId: INT [PK], firstName: VARCHAR(60), lastName: VARCHAR(60), heightInches: INT, bodyWeightLbs: INT)
- Team(teamId: INT [PK], name: VARCHAR(80), city: VARCHAR(60))
- Game(gameId: INT [PK], date: DATETIME, arenaId: INT [FK to Arena.arenaId])
- Official(officialId: INT [PK], fullName: VARCHAR(100))
- Arena(arenaId: INT [PK], name: VARCHAR(100), city: VARCHAR(60), state: VARCHAR(30))

- PlayerGameStats(gameId: INT [PK] [FK to Game.gameId], playerId: INT [PK] [FK to Player.playerId], teamId: INT [FK to Team.teamId], minutes: DECIMAL(4,1), points: INT, rebounds: INT, assists: INT, steals: INT, blocks: INT, turnovers: INT, fg_percent: DECIMAL(5,4), three_percent: DECIMAL(5,4), ft_percent: DECIMAL(5,4))
- TeamGameStats(gameId: INT [PK] [FK to Game.gameId], teamId: INT [PK] [FK to Team.teamId], isHome: BOOLEAN, teamScore: INT, rebounds: INT, assists: INT, steals: INT, blocks: INT, turnovers: INT, fg_percent: DECIMAL(5,4), three_percent: DECIMAL(5,4), ft_percent: DECIMAL(5,4))
- GameOfficial(gameId: INT [PK] [FK to Game.gameId], officialId: INT [PK] [FK to Official.officialId])